

פרויקט 1

מגישה: מרים חאוי

מרצה: פרופ' חדד רפי

תאריך הגשה: 17.5.2026

A.1 חלוקת האות והצגת גרפית

בסעיף זה חילקנו את האות המוקלט לשני חלקים עיקריים בהתאם לזמן: המקטע הראשון, בין 0 ל - 9 שניות, הוגדר כרעש (baseline). המקטע השני, בין 12 ל - 30 שניות, הוגדר כאות הגירוי (stimulus) המכיל גם את התגובה לגירוי וגם רעש.

החלוקה בוצעה על סמך קצב הדגימה (50Hz) כלומר כל שנייה מכילה 50 דגימות, וכך ניתן להמיר זמנים לאינדקסים במערך הנתונים. בנוסף, בנינו ציר זמן לכל מקטע באמצעות פונקציה הממירה אינדקסים לשניות.

הצגת שלושת הגרפים:

1. גרף הרעש (0-9 שניות) - מציג את התנהגות הרעש בלבד.

קטע הקוד והצגת הגרף:

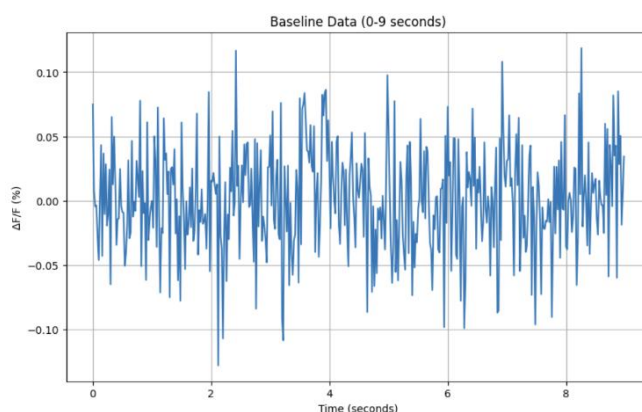
```
# sampling rate
HZ = 50

#first 9 seconds as baseline (noise)
baseline_end_time = 9
baseline_df = df.iloc[0 : baseline_end_time * HZ]

#signal starting from 12 seconds (stimulus part)
stim_start_time = 12
stim_df = df.iloc[stim_start_time * HZ : ]

#time axis for plotting
def get_time_axis(data_frame, sampling_rate, start_offset=0):
    return (np.arange(len(data_frame)) / sampling_rate) + start_offset

#signals as numpy arrays
stim_signal = stim_df['ΔF/F (%)'].values # signal + noise
noise_signal = baseline_df['ΔF/F (%)'].values # noise only
```

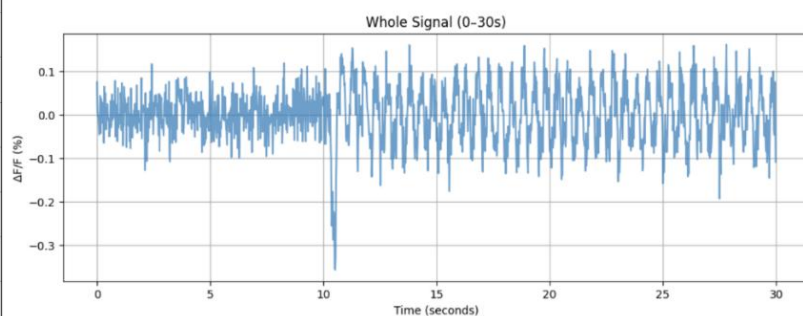
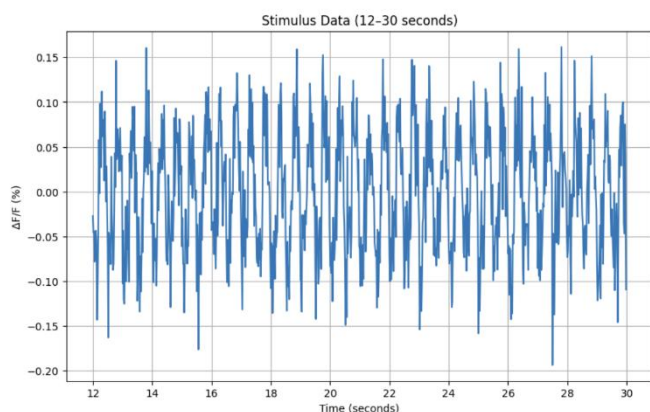


2. גרף הגירוי (12-30 שניות) - מציג את האות בזמן הפעלת הגירוי.

3. גרף האות המלא (0-30 שניות) - מציג את כל האות ברצף, ומאפשר לראות בצורה כוללת את ההבדל בין

אזור הרעש לבין אזור הגירוי.

הצגת הגרפים:



B.1 דגימה מחדש והשפעת תדר הדגימה על איכות האות

האות המקורי נדגם בתדר של 50Hz. כדי לבצע דגימה מחדש לתדרים 10Hz, 25Hz ו-3.33Hz, חישבנו את גודל הצעד (step) לפי היחס בין תדר הדגימה המקורי לתדר היעד: עבור 25Hz דגימה כל נקודה שנייה ($50/25 = 2$), עבור 10Hz כל נקודה חמישית ($50/10 = 5$) ועבור $\approx 3.33\text{Hz}$ דגימה כל נקודה 15 ($50/15 \approx 3.33$). לאחר הדגימה מחדש, נבנה ציר זמן חדש לכל אות בהתאם לתדר הדגימה בפועל, על מנת לשמור על ייצוג נכון של הזמן.

מהתוצאות ניתן לראות כי עבור תדרים של 25Hz ו-10Hz האות שומר על צורתו הכללית, כלומר רוב המידע נשמר.

קטע הקוד והצגת הגרפים:

1.B Manual Signal Resampling

```
#target sampling frequencies
target_frequencies = [25, 10, 3.33]

# use only the stimulus part
stimulus_part = stim_signal

for f_new in target_frequencies:

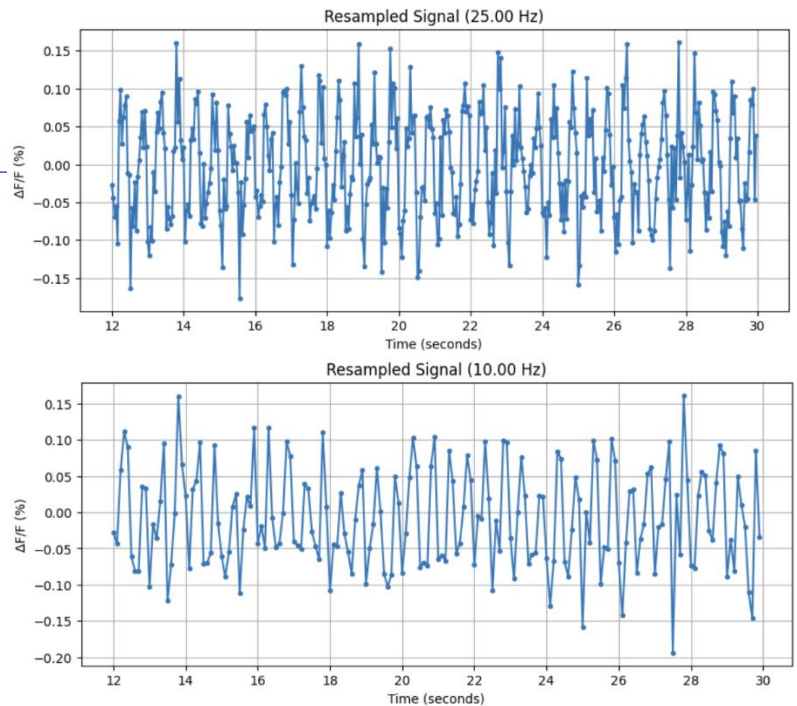
    # calculate step size (how many samples to skip)
    step = round(HZ / f_new)

    # actual sampling rate after rounding
    actual_fs = HZ / step

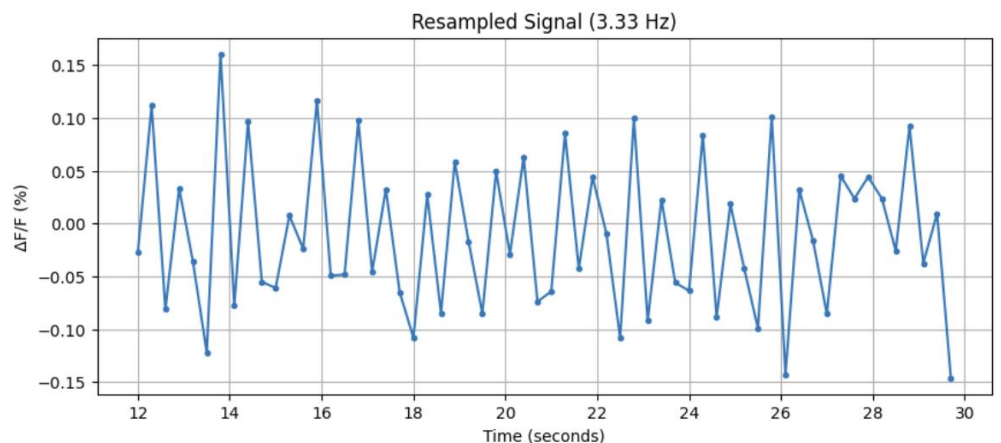
    # downsample by taking every 'step'
    resampled_data = stimulus_part[::step]

    # create new time axis
    time_resampled = np.arange(len(resampled_data)) / actual_fs + stim_start_time

    # plot the resampled signal
```



לעומת זאת, כאשר תדר הדגימה יורד ל- $\approx 3.33\text{Hz}$ מתקבלת ירידה ברזולוציית הזמן, וקשה יותר לייצג שינויים מהירים באות:



בהתאם למשפט נייקוויסט, יש צורך בתדר דגימה הגבוה לפחות פי שניים מהתדר המקסימלי באות. בתדר הנמוך $\approx 3.33\text{Hz}$ תנאי זה כמעט ואינו מתקיים, ולכן מתקבל איבוד מידע ואף אליאסינג, תופעה זו מתבטאת בכך שהאות נראה מדולל יותר ופחות מדויק, דבר הפוגע באמינות הניתוח.

C.1 השפעת תדר הדגימה ואליאסינג על זיהוי התדר

עבור כל תדר דגימה ביצענו דגימה מחדש (Downsampling) והערכנו את התדר הנצפה באמצעות ספירת פיקים לאורך 12 עד 30 שניות של הגירוי. בנוסף, השווינו את התוצאות לחישוב של תדר האליאסינג: $f_{alias} = |f - n \cdot f_s|$. לפי התוצאות, ניתן לראות כי עבור תדרי דגימה של 10 ו-25 הרץ, מתקיים תנאי נייקוויסט $f_s > 2 \cdot f_{signal}$, לכן אין אליאס, ומספר הפיקים בגרף מתאים לתדר המקורי של האות.

לעומת זאת, עבור תדר דגימה של 3.33 Hz, תנאי נייקוויסט אינו מתקיים, משום ש- $3.33 < 4 \text{ Hz}$, וכתוצאה מכך

נוצר אליאס. לפי החישוב מתקבל: $f_{alias} = |2 - 3.33| = 1.33 \text{ Hz}$

תוצאה זו מתאימה למספר הפיקים שנצפה בפועל בגרף (כ-24 פיקים לאורך 18 שניות), כלומר התדר שנמדד נמוך יותר מהתדר המקורי. מכאן ניתן לראות שבדגימה לא מספקת, האות משתנה ומקבל ייצוג שגוי של תדר נמוך יותר, למרות שהתדר המקורי לא השתנה.

קטע הקוד והתוצאות:

```
f_signal = 2.0
duration = 18.0

print(f"{'Actual Fs':<12} | {'Calculation (N)':<18} | {'f_alias (Hz)':<15} | {'Exp. Peaks'}")
print("-" * 65)

for f_target in target_frequencies:

    # compute downsampling step size
    step = round(HZ / f_target)

    # actual sampling frequency after rounding
    actual_fs = HZ / step

    # check Nyquist condition
    if actual_fs > 2 * f_signal:
        f_alias = f_signal
        calc_str = "No Aliasing"
    else:
        # compute possible aliasing candidates
        possible_aliases = []
        for n in [1, 2, 3]:
            val = abs(f_signal - n * actual_fs)
            possible_aliases.append(val)

        # choose smallest alias frequency
        f_alias = min(possible_aliases)

        # index of selected n
        n_used = possible_aliases.index(f_alias) + 1

        calc_str = f"|{f_signal} - {n_used}*{actual_fs:.2f}|"

    # expected number of peaks over full duration
    expected_peaks = f_alias * duration

    print(f"{'actual_fs':<12.2f} | {'calc_str':<18} | {'f_alias':<15.2f} | {'expected_peaks':.1f}")
```

Actual Fs	Calculation (N)	f_alias (Hz)	Exp. Peaks
25.00	No Aliasing	2.00	36.0
10.00	No Aliasing	2.00	36.0
3.33	2.0 - 1*3.33	1.33	24.0

D.1 קוונטיזציה ומספר הביטים המינימלי הנדרש

בסעיף זה, עלינו לחשב כמה ביטים נדרשים כדי להבחין בשינוי קטן של 0.1% בעוצמת האות. קודם כל נמיר את האחוז לשבר עשרוני, $0.1\% = 0.001$: כלומר אנחנו רוצים שהמערכת תוכל להבדיל בין רמות שגודלן בערך אחד מתוך אלף.

לאחר מכן נשתמש בעובדה שמערכת עם n ביטים יכולה לייצג 2^n רמות שונות. לכן, נחפש את הערך המינימלי של n שמקיים $2^n \geq 1000$. בקוד נחשב זאת בעזרת לוגריתם בבסיס 2, כלומר $\log_2(1/0.001)$, ואז ניקח עיגול כלפי מעלה כדי לקבל מספר שלם של ביטים. הצגת הקוד והתוצאה:

```
s
def calculate_min_bits(resolution_percent):
    resolution_fraction = resolution_percent / 100
    n_bits = np.ceil(np.log2(1 / resolution_fraction))
    return int(n_bits)

required_bits = calculate_min_bits(0.1)

print("\nQuantization Analysis:")
print(f"To distinguish a 0.1% change, we need at least {required_bits} bits.")

Quantization Analysis:
To distinguish a 0.1% change, we need at least 10 bits.
```

כך התקבלה התוצאה שנדרשים לפחות 10 ביטים כדי שהמערכת תוכל להבחין בשינוי של 0.1% בצורה אמינה.

1. חישוב SNR בשתי שיטות והשוואת התוצאות

בסעיף זה חישבנו את יחס האות לרעש (SNR_{rms}) בשתי שיטות שונות שנלמדו בכיתה.

בשיטה הראשונה "Power Subtraction Method" - חישבנו את עוצמת הרעש מתוך מקטע ה - baseline:

$$P_{noise} = \text{mean}(noise^2)$$

ואת העוצמה הכוללת של מקטע הגירוי: $P_{recorded} = \text{mean}(signal^2)$

מכיוון שהאות המוקלט כולל גם את האות עצמו וגם רעש, חיסרנו את הרעש כדי לקבל את עוצמת האות הנקי:

$$P_{signal} = P_{recorded} - P_{noise}$$

$$SNR_{rms} = \sqrt{\frac{P_{signal}}{P_{noise}}}$$

ולאחר מכן חישבנו:

בשיטה השנייה "Amplitude Method" - חישבנו את היחס בין ממוצע האות במקטע הגירוי לבין סטיית התקן של

$$SNR = \frac{\mu_{signal}}{\sigma_{noise}}$$

קטע הקוד והתוצאות:

```
P_noise = np.mean(noise_signal**2)
P_recorded = np.mean(stim_signal**2)

P_signal_pure = P_recorded - P_noise
if P_signal_pure < 0: P_signal_pure = 0

snr_rms_way1 = np.sqrt(P_signal_pure / P_noise)

snr_rms_way2 = np.mean(stim_signal) / np.std(noise_signal)

print(f"Method 1 - Power Subtraction: {snr_rms_way1:.4f}")
print(f"Method 2 - Amplitude: {snr_rms_way2:.4f}")
```

```
Method 1 - Power Subtraction: 1.3376
Method 2 - Amplitude: -0.0231
```

התקבלו תוצאות שונות בין שתי השיטות בגלל אופי האות. האות במקטע הגירוי הוא אות תנודתי סביב האפס, ולכן בשיטת האמפליטודה התנודות החיוביות והשליליות נוטות לקזז זו את זו. כתוצאה מכך התקבל ערך קטן מאוד ואף שלילי, שאינו מייצג היטב את עוצמת הפעילות האמיתית של האות. לעומת זאת, בשיטה הראשונה התקבל ערך חיובי, משום שהעלאה בריבוע הופכת את כל הערכים

לחיוביים ולכן מודדת את האנרגיה הכוללת של הסיגנל ללא תלות בכיוון התנודה.

2. השפעת ממוצע בלוקים על הפחתת רעש ושיפור SNR

בסעיף זה ביצענו הפחתת רעש באמצעות Block Averaging, כלומר חלוקה של האות למקטעים (באורך 2, 3 ו- 5 דגימות) וחישוב ממוצע לכל מקטע במקום חלון נע. בקוד עצמו נחתכו הדגימות כך שהאורך מתחלק ב - n, ולאחר מכן בוצע reshape וחישוב ממוצע לכל בלוק. כך מתקבל אות עם תדר אפקטיבי נמוך יותר, שבו תנודות מהירות (בעיקר רעש) מוחלקות.

לאחר מכן חושב ה - SNR לכל גרסה של האות באמצעות פונקציה מבוססת RMS, שמחשבת את עוצמת האות והרעש דרך ממוצע ריבועי והפרדתם באמצעות חיסור ההספקים. התוצאות מראות שה - SNR גדל ככל ש - n עולה, כלומר ככל שהחלקה חזקה יותר.

```
def calc_snr(signal_samples, noise_samples):
    # compute signal and noise power (RMS based)
    S2 = np.mean(signal_samples**2)
    N2 = np.mean(noise_samples**2)

    # extract pure signal power by subtracting noise
    P_signal = S2 - N2
    P_signal = max(P_signal, 0)

    # compute SNR (avoid division by zero)
    return np.sqrt(P_signal / N2) if N2 > 0 else 0

def block_average(data, n):
    # trim data so it can be divided into equal blocks
    length = len(data) // n
    trimmed = data[:length * n]

    # reshape into blocks and average each block
    return trimmed.reshape(-1, n).mean(axis=1)

# compute original SNR
original_snr = calc_snr(stim_signal, noise_signal)

factors = [2, 3, 5]

plt.figure(figsize=(12, 6))

# time axis for original signal
time_orig = (np.arange(len(stim_signal)) / HZ) + 12
plt.plot(time_orig, stim_signal, alpha=0.3, label='Original (50Hz)')

for n in factors:

    # apply block averaging to signal and noise
    signal_avg = block_average(stim_signal, n)
    noise_avg = block_average(noise_signal, n)

    # compute SNR after averaging
    snr_new = calc_snr(signal_avg, noise_avg)

    # adjust time axis according to new sampling rate
    time_avg = np.arange(len(signal_avg)) * (n / HZ) + 12

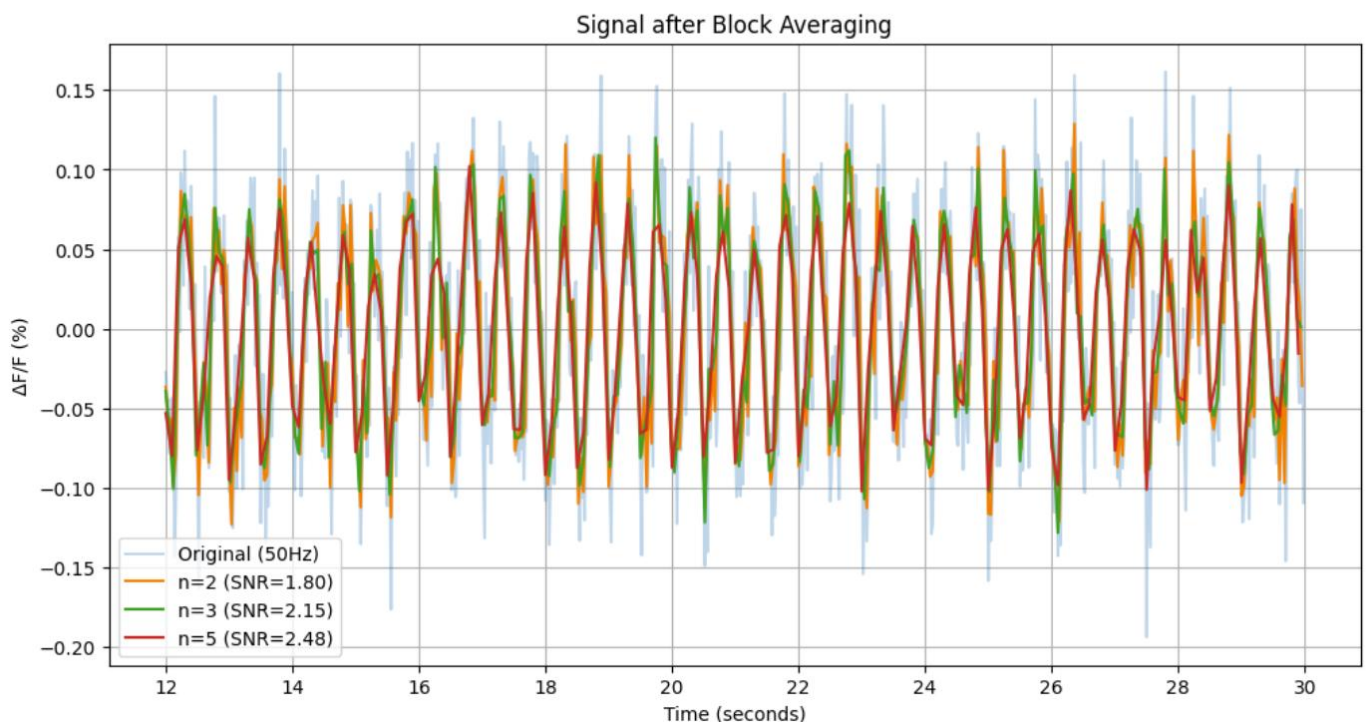
    # plot averaged signal
    plt.plot(time_avg, signal_avg, label=f'n={n} (SNR={snr_new:.2f})')

    print(f"After averaging every {n} points: SNR = {snr_new:.4f}")
```

מהגרף ניתן לראות שהאות המקורי (בצבע תכלת) רועש ותנודתי יותר, בעוד שלאחר ממוצע עבור $n=2,3,5$ האות נעשה חלק יותר בהדרגה. לפיכך, ככל ש- n גדול יותר מתקבלת החלקה חזקה יותר, שמפחיתה רעש אך גם גורמת לאיבוד פרטים מהירים באות.

הצגת הגרף:

After averaging every 2 points: SNR = 1.8021
 After averaging every 3 points: SNR = 2.1533
 After averaging every 5 points: SNR = 2.4823



3. סינון רעש באמצעות ממוצע 20 דגימות

בסעיף זה ביצענו הפחתת רעש באמצעות Block Averaging, כלומר חלוקה של האות המקורי לבלוקים של 20 דגימות וחישוב ממוצע לכל בלוק.

```
n = 20

# compute block averaged signal
signal_avg_20 = block_average(stim_signal, n)

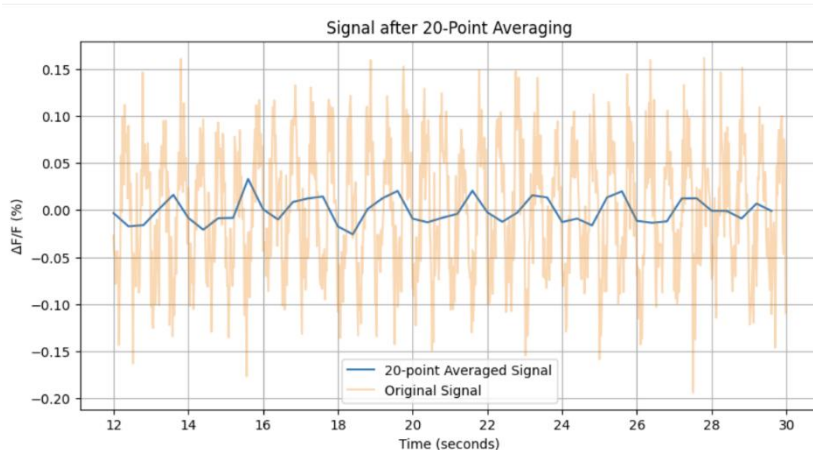
# effective sampling rate after averaging
new_fs = HZ / n

# time axis for the downsampled signal
time_avg_20 = np.arange(len(signal_avg_20)) * (n / HZ) + 12

plt.figure(figsize=(10, 5))

# plot averaged smoothed signal
plt.plot(time_avg_20, signal_avg_20, label='20-point Averaged Signal')

# original signal for comparison
time_orig = (np.arange(len(stim_signal)) / HZ) + 12
plt.plot(time_orig, stim_signal, alpha=0.3, label='Original Signal')
```



בקוד, הגדרנו $n=20$, ולאחר מכן האות נחתך כך שמספר הדגימות יתאים לחלוקה מלאה לבלוקים. לאחר מכן, בוצע reshape למבנה של בלוקים חישוביים, וחישוב ממוצע לכל בלוק יצר אות חדש בקצב דגימה נמוך יותר.

בגרף ניתן לראות שהאות הממוצע (כחול) חלק משמעותית לעומת האות המקורי (צהוב). הסיבה לכך שכבר לא ניתן לראות את התנודות המקוריות היא שהממוצע על פני 20 נקודות מבצע low pass effect, כלומר תנודות מהירות נחלשות מאוד ואף מתבטלות, בעוד שרק השינויים האיטיים נשמרים. בנוסף, הקטנת הרזולוציה הזמנית גורמת לאובדן מידע בתדרים גבוהים, ולכן האוסצילציות המקוריות נעלמות ויזואלית.

4. החלקת אות באמצעות גרעין מלבני כפול

בסעיף זה ביצענו החלקת אות באמצעות קונבולוציה עם גרעין מלבני (Rectangular Kernel) בגודל 3, פעמיים ברצף. בתחילה הגדרנו גרעין ממוצע שבו כל שלוש הנקודות מקבלות משקל שווה, ולאחר מכן בוצעה קונבולוציה של האות עם הגרעין פעם נוספת על האות שכבר הוחלק. פעולה זו יוצרת אפקט של low pass, אשר מדכא תנודות מהירות באות ומפחית רעש. הצגת הקוד:

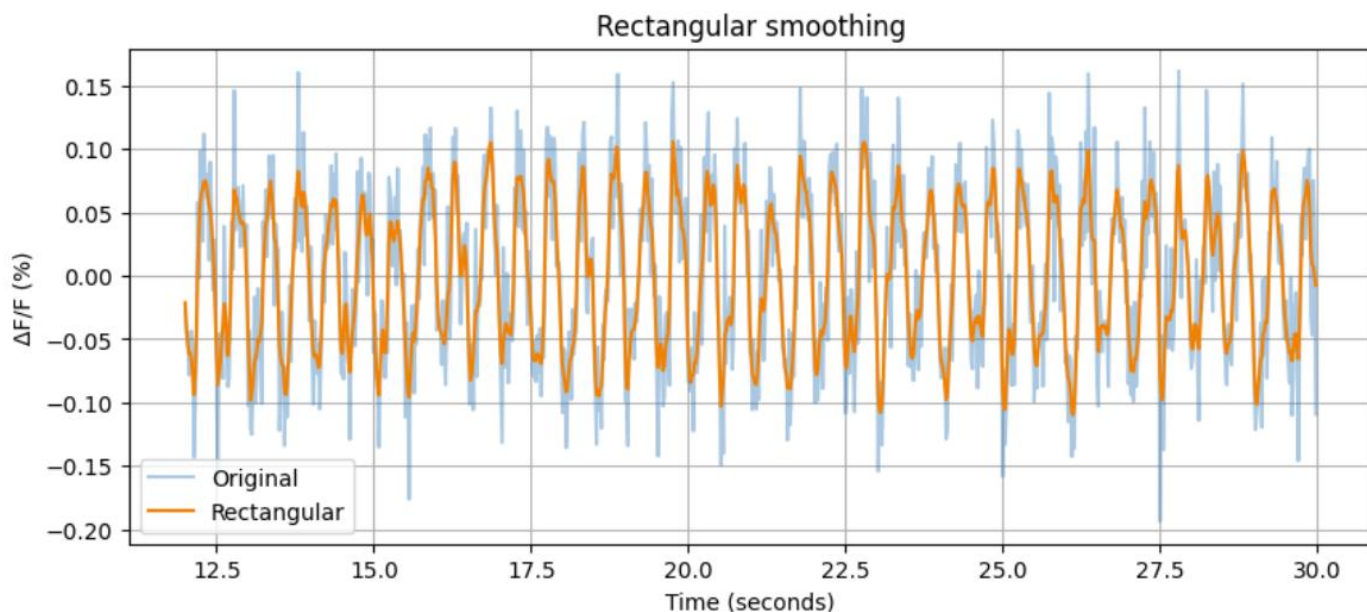
```
# rectangular smoothing kernel
rect_kernel = np.ones(3)

# normalize kernel so sum = 1
rect_kernel = rect_kernel / np.sum(rect_kernel)

# first smoothing pass (moving average)
rect_once = np.convolve(stim_signal, rect_kernel, mode='same')

# second smoothing pass
rect_twice = np.convolve(rect_once, rect_kernel, mode='same')
```

מהגרף ניתן לראות כי לאחר שתי פעולות החלקה (כתום), האות חלק משמעותית ביחס לאות המקורי (תכלת). התנודות המהירות כמעט ונעלמות, בעוד שהמגמה הכללית של האות נשמרת. עם זאת, ההחלקה הכפולה גורמת גם לאובדן פרטים מהירים באות, ולכן יש כאן איזון בין הפחתת רעש לבין איבוד רזולוציה זמנית.



5. החלקת אות באמצעות גרעין משולש

בסעיף זה ביצענו החלקת אות באמצעות גרעין משולש (Triangular Kernel) בגודל 5. הגרעין מוגדר כך שערכים קרובים למרכז מקבלים משקל גבוה יותר, בעוד שערכים בקצוות מקבלים משקל נמוך יותר, ולאחר מכן הוא מנורמל כך שסכום המשקלים יהיה 1. לאחר בניית הגרעין ביצענו קונבולוציה עם האות המקורי, וכך התקבל אות מוחלק.

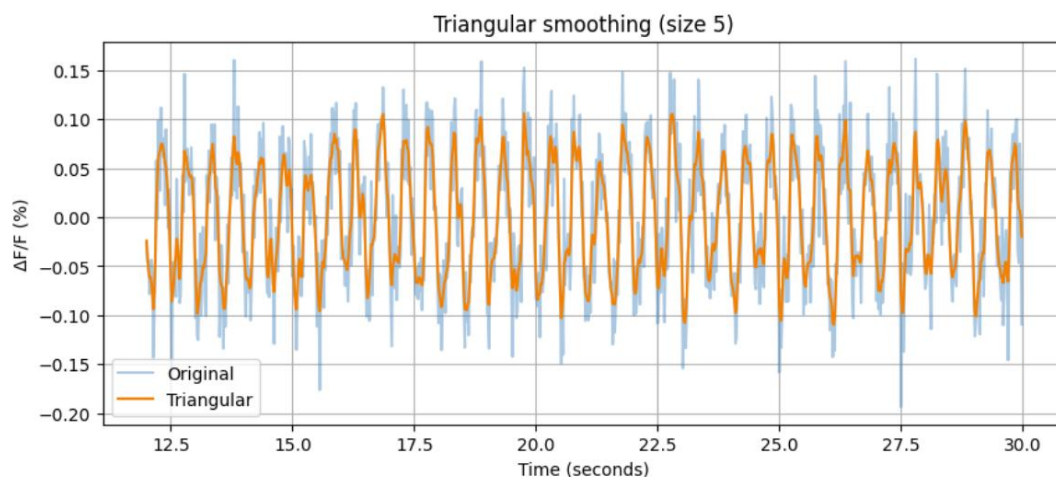
קטע הקוד:

```
# define triangular kernel (higher weight in center)
tri_kernel = np.array([1, 2, 3, 2, 1])

# normalize kernel so sum = 1
tri_kernel = tri_kernel / tri_kernel.sum()

# apply convolution for smoothing
tri_smoothed = np.convolve(stim_signal, tri_kernel, mode='same')
```

מהגרף ניתן לראות שהאות לאחר ההחלקה (כחול) חלק יותר מהאות המקורי (תכלת), אך עדיין שומר על המבנה הכללי של השינויים באות. הצגת הגרף:



6. השוואת שיטות החלקה

```
plt.figure(figsize=(12,6))

plt.title("Comparison of smoothing methods")
plt.xlabel("Time (seconds)")
plt.ylabel("ΔF/F (%)")

# original noisy signal
plt.plot(time_stim, stim_signal, alpha=0.4, label="Original")

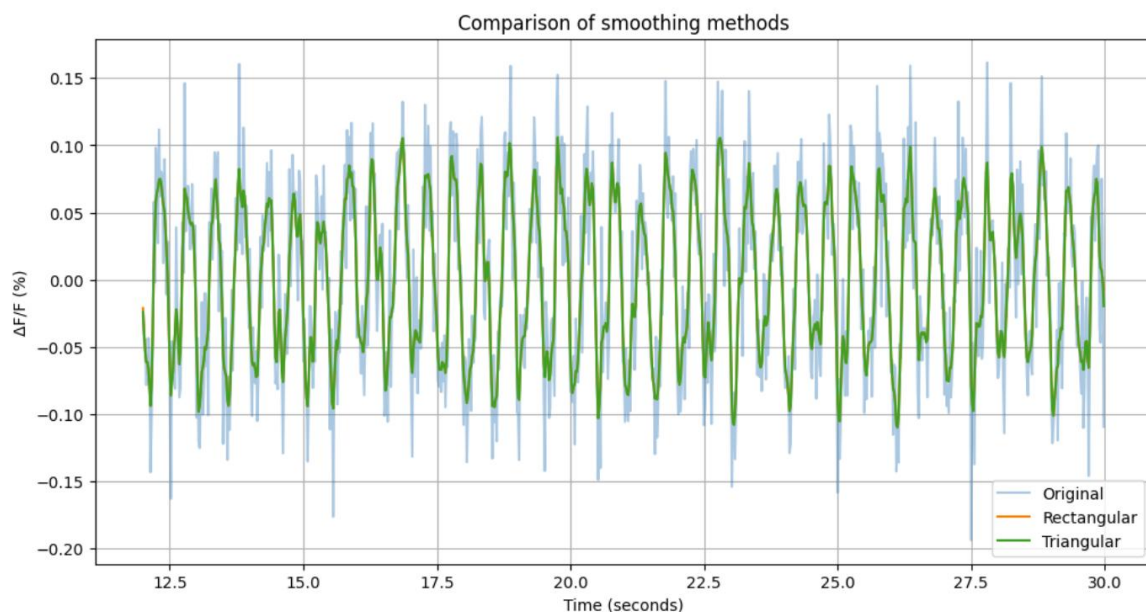
# rectangular smoothing result
plt.plot(time_stim, rect_twice, label="Rectangular")

# triangular smoothing result
plt.plot(time_stim, tri_smoothed, label="Triangular")

plt.grid(True)
plt.legend()
plt.show()
```

בסעיף זה השווינו בין שלושה אותות: האות המקורי, האות לאחר החלקה עם גרעין מלבני, והאות לאחר החלקה עם גרעין משולש. בקוד בוצעה הצגה משותפת של שלושת האותות על אותו גרף על מנת לאפשר השוואה ישירה בין שיטות הסינון. הצגת הקוד:

ניתן לראות כי הגרפים של ההחלקה המלבנית הכפולה ושל ההחלקה המשולשת חופפים לחלוטין, כך שבפועל מתקבל קו אחד בלבד (ירוק), בעוד שהאות המקורי נותר תנודתי יותר. הסיבה לכך היא שלפי תכונת הקיבוציות של הקונבולוציה מתקיים: $(x * w_1) * w_2 = x * (w_1 * w_2)$. כלומר, החלקה של אות פעמיים ברצף שקולה להחלקה אחת עם קרנל חדש שמתקבל מהקונבולוציה בין שני הקרנלים. במקרה שלנו, ביצענו פעמיים החלקה עם הקרנל המלבני $[1, 1, 1]$. כאשר מבצעים קונבולוציה של הקרנל הזה עם עצמו מתקבל הקרנל המשולש $[1, 2, 3, 2, 1]$, שהוא בדיוק הקרנל שבו השתמשנו בהחלקה המשולשת. לכן שתי השיטות מבצעות בפועל את אותה פעולת סינון על האות, ולכן מתקבלים גרפים זהים לחלוטין.



7. סינון Low Pass

פעולת ההחלקה (Smoothing), נחשבת ל - Low Pass Filter, מכיון שהיא מדכאת תנודות מהירות ורעש (תדרים גבוהים) ומאפשרת מעבר של תדרים נמוכים בלבד, כלומר השינויים האיטיים של האות. מבחינה מתמטית, פעולת ההחלקה מבצעת ממוצע בין כל נקודה לבין הנקודות שסביבה. כאשר מחשבים ממוצע, ערכים חיוביים ושליילים של תנודות מהירות נוטים לקזז זה את זה, ולכן הרעש קטן. לעומת זאת, שינויים איטיים באות כמעט ואינם משתנים במהלך הממוצע ולכן נשמרים. לכן ההחלקה מחלישה רכיבים בעלי תדר גבוה ושומרת על רכיבים בעלי תדר נמוך ומתקבל אות חלק יותר שמייצג את המגמה הכללית בלבד.

Mean Firing Rate A.1

בסעיף זה חישבנו את קצב הירי הממוצע (Mean Firing Rate) של כל נוירון לאורך כל הטריילים. תחילה זוהה סוג הנוירון (A - E) מתוך הקובץ. לאחר מכן, עבור כל טרייל נספר מספר הספייקים הכולל על ידי סכימת כל ערכי ה - 1 בשורת הנתונים, כאשר כל 1 מייצג ירי של הנוירון. את מספר הספייקים חילקנו במשך ההקלטה (30 שניות) כדי לקבל את קצב הירי ביחידות של Hz, כלומר מספר ספייקים לשנייה.

לבסוף, בוצע ממוצע של קצב הירי על פני כל הטריילים של כל נוירון באמצעות groupby, וכך התקבל קצב הירי הממוצע עבור כל נוירון. התוצאות מאפשרות להשוות בין רמות הפעילות של הנוירונים השונים ולזהות אילו נוירונים יורים בתדירות גבוהה יותר. הצגת הקוד והתוצאות:

```
trial_duration_sec = 30

# extract neuron label (A-E)
df3['neuron'] = df3['trial_neuron'].str.extract(r'([A-E])', expand=False)

# count spikes in each trial and convert to firing rate (Hz)
df3['trial_fr'] = (df3.drop(columns=['trial_neuron', 'neuron']).sum(axis=1) / trial_duration_sec)

# compute mean firing rate across trials for each neuron
fr_results = (df3.groupby('neuron')['trial_fr'].mean().to_frame(name='Mean Firing Rate (Hz)'))

print(fr_results)
```

Mean Firing Rate (Hz)	
neuron	
A	11.670333
B	42.550000
C	100.000000
D	49.984333
E	28.679000

Fano Factor B.1

בסעיף זה חישבנו את ה - Fano Factor עבור כל נוירון. החישוב בוצע על חלון זמן קבוע של שנייה אחת, כלומר 1000 דגימות בקצב דגימה של 1000Hz. בקוד בחרנו את 1000 העמודות הראשונות המייצגות את השנייה הראשונה של כל טרייל. לאחר מכן נספר מספר הספייקים בכל טרייל באמצעות סכימת ערכי ה - 1, ונשמר כ - spike_count. עבור כל נוירון חושבו הממוצע והשונות של מספר הספייקים בין הטריילים, ולבסוף חושב ה - Fano

```
# take 1 second window (1000 samples)
spike_data = df3.iloc[:, 1:1001]

# count spikes per trial
df3['spike_count'] = spike_data.sum(axis=1)

# calculate FF per neuron
ff_results = df3.groupby('neuron')['spike_count'].agg(['mean', 'var'])

ff_results['Fano_Factor'] = ff_results['var'] / ff_results['mean']

print(ff_results['Fano_Factor'])
```

$$\text{Factor לפי: } FF = \frac{\text{Var}}{\text{Mean}}$$

הצגת הקוד והתוצאות:

neuron	
A	0.986133
B	0.574943
C	0.000000
D	0.010218
E	6.653800

Name: Fano_Factor, dtype: float64

ניתוח התוצאות מראה כי ניתן להשתמש ב - Fano Factor כדי לאפיין את סוג פעילות הירי של כל נוירון. כאשר נוירון A התקבל עם $FF \approx 0.99$, ערך הקרוב מאוד ל - 1, ולכן הוא מתאים למודל פואסוני שבו הירי אקראי ובלתי תלוי. נוירון B התקבל עם $FF \approx 0.57$, כלומר שונות נמוכה מהמוצע, דבר המעיד על ירי יחסית סדיר. נוירונים C ו - D התקבלו עם ערכים קרובים מאוד ל - 0 ($FF = 0$ ו - $FF \approx 0.01$), ולכן הם מציגים ירי כמעט קבוע לחלוטין בין הניסויים, עם שונות מזערית במספר הספייקים. לעומת זאת, נוירון E התקבל עם $FF \approx 6.65$, ערך גבוה המעיד על ירי בצורת Bursting, כלומר תקופות של פעילות מוגברת לצד תקופות שקטות, דבר שיוצר שונות גבוהה במיוחד בין החלונות השונים.

A – Fano Factor C.1 – נוירון

כפי שהוסבר בסעיף הקודם, נוירון A הינו מסוג מודל פואסוני עם FF קרוב ל -1.

B, A – ISI distribution D.1 – נוירונים

בסעיף זה חישבנו והצגנו את התפלגות זמני ה - ISI עבור הנוירונים A ו-B. בקוד בודדנו תחילה כל הספייקים של כל נוירון, ולאחר מכן חישבנו את המרווחים בין ספייקים עוקבים באמצעות np.diff. מכיוון שתדר הדגימה הוא 1000 Hz, כל מרחק בין אינדקסים הומר לשניות על ידי כפל ב-1/1000. לבסוף, כל מרווחי הזמן מכל הניסויים אוחדו ונבנה עבורם היסטוגרמה המתאר את התפלגות ה - ISI. הצגת הקוד:

```
new_HZ = 1000

# keep only spike columns
spike_data = df3.drop(columns=['trial_neuron', 'neuron', 'trial_fr', 'spike_count'])

# get trials of neuron A and B
neuron_A_trials = spike_data[df3['neuron'] == 'A'].values
neuron_B_trials = spike_data[df3['neuron'] == 'B'].values

# function to plot ISI distribution
def ISI_plot(data, title):

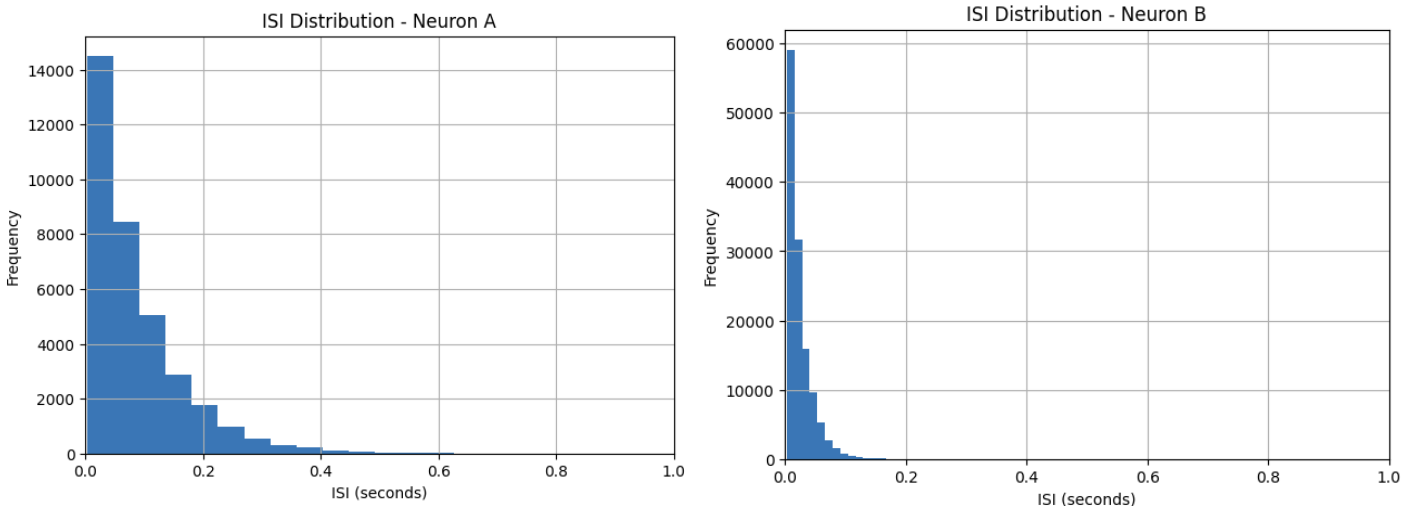
    all_distances = []

    # time per sample
    bin_size = 1 / new_HZ

    for trial in data:

        # indices where spike = 1
        spike_positions = [i for i, val in enumerate(trial) if val]
        # differences between consecutive spikes
        distances = np.diff(spike_positions)
        # convert from samples to seconds
        distances = distances * bin_size
        # add to list
        all_distances += list(distances)
```

מהתוצאות ניתן לראות כי נירון A מציג התפלגות ISI רחבה ואקספוננציאלית, עם ריבוי של מרווחים קצרים ומיעוט של מרווחים ארוכים. צורה זו אופיינית לירי פואסוני, שבו ההסתברות לספייק אינה תלויה בזמן שחלף מאז הספייק הקודם, ולכן מתקבלת שונות גבוהה בזמני הירי. לעומת זאת, נירון B מציג התפלגות צרה ומרוכזת יותר סביב ערך ISI מסוים, המתבטא כפיק בגרף. דפוס זה מעיד על ירי סדיר (Regular firing), שבו הנירון שומר על קצב ירי יציב יחסית עם שונות נמוכה במרווחי הזמן בין הספייקים. הצגת הגרפים:



E, D, C, A – Raster Plot E.1

```
neurons_to_plot = ['A', 'C', 'D', 'E']

for neuron in neurons_to_plot:

    # get trials of current neuron
    neuron_trials = spike_data[df3['neuron'] == neuron].values

    plt.figure(figsize=(10, 5))

    # go over all trials
    for trial_index, trial in enumerate(neuron_trials):

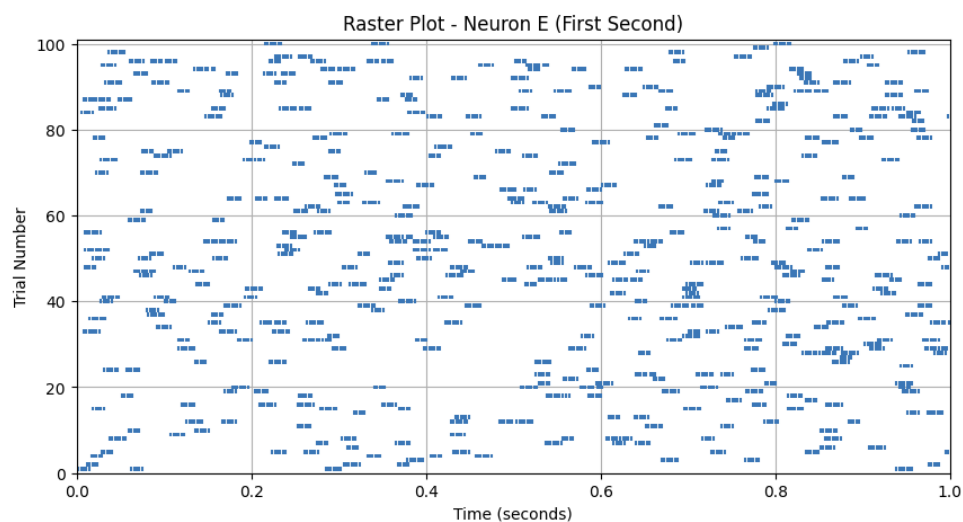
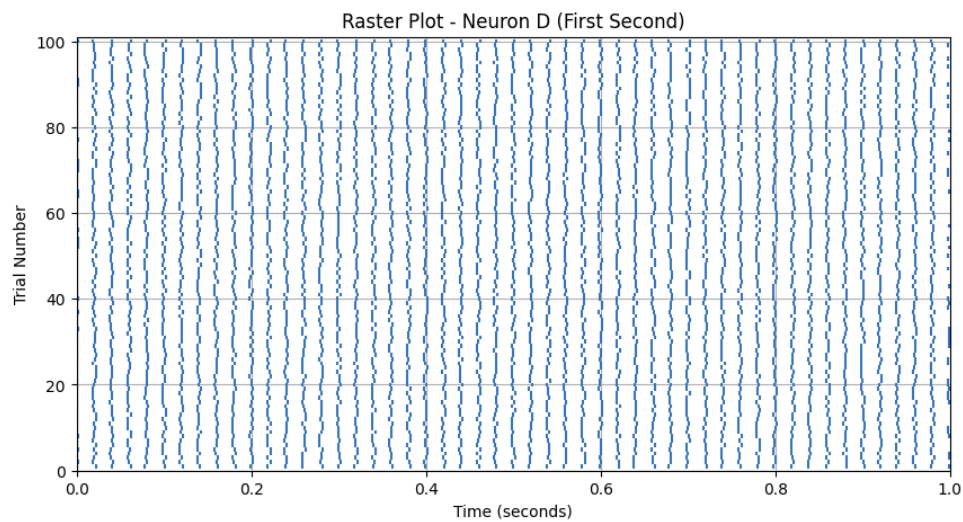
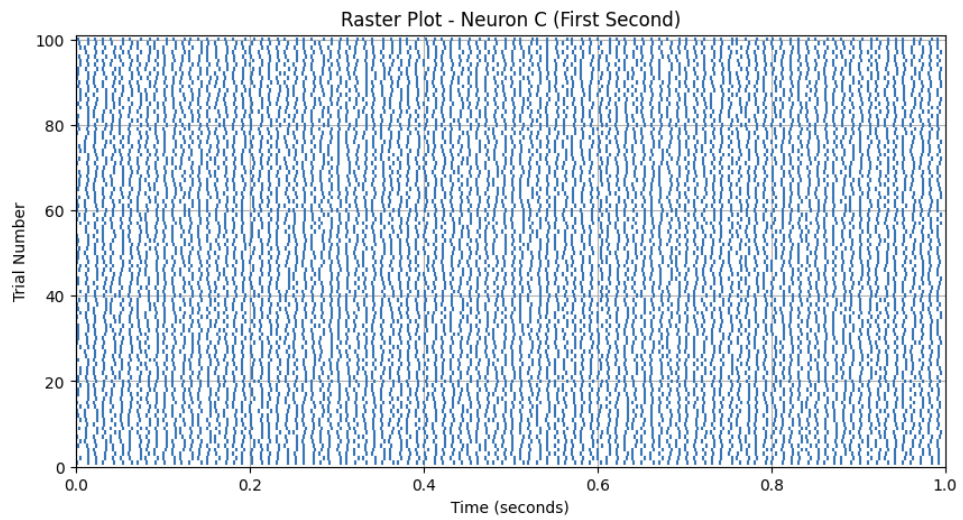
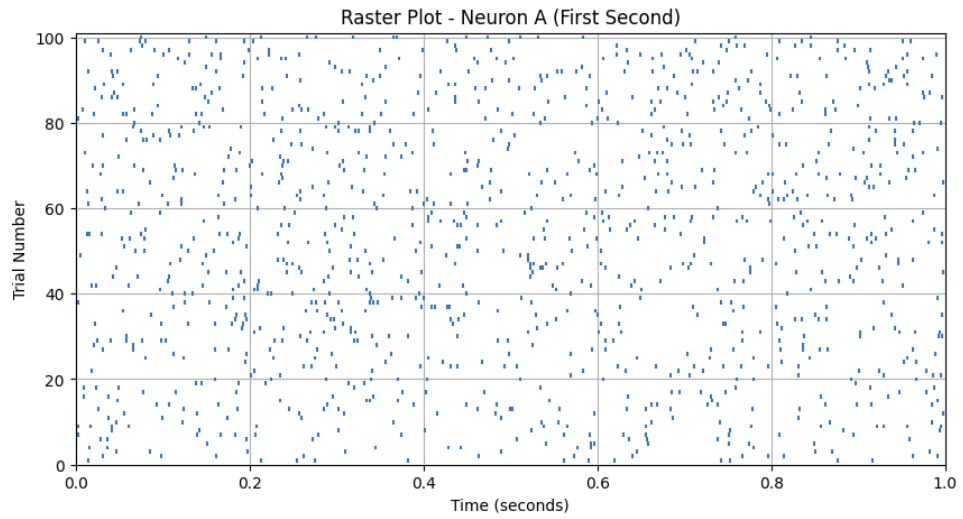
        # find spike locations
        spike_times = np.where(trial[:new_HZ] == 1)[0]

        # convert to seconds
        spike_times = spike_times / new_HZ

        # draw raster lines
        plt.vlines(spike_times, trial_index + 0.5, trial_index + 1.5)
```

בסעיף זה יצרנו Raster Plot עבור הנירונים A, C, D ו-E, תוך התמקדות בשנייה הראשונה של ההקלטה. בקוד עברנו על כל נירון בנפרד, בחרנו את כל הניסויים שלו, ולאחר מכן עבור כל trial איתרנו את זמני הספייקים באמצעות np.where. מכיוון שתדר הדגימה הוא 1000Hz, האינדקסים הומרו לשניות על ידי חלוקה ב-1000. לבסוף, כל ספייק הוצג כקו אנכי (vlines) במיקום הזמן המתאים ובשורת ה-trial המתאימה, וכך התקבל ייצוג חזותי של דפוס הירי לאורך הניסויים.

מהגרפים ניתן לראות דפוסי ירי שונים בין הנירונים. נירון A מציג ירי אקראי יחסית ללא מבנה קבוע, דבר המתאים להתנהגות פואסונית. נירונים C ו-D מציגים ירי סדיר ועקבי, כאשר הספייקים מופיעים כמעט באותם זמנים בכל trial, ולכן מתקבלים קווים אנכיים מסודרים. נירון D מציג מעט יותר שונות בזמן הירי. לעומת זאת, נירון E מציג דפוס של - burst firing קבוצות של ספייקים צפופים המופיעים בזמנים מסוימים לצד תקופות שקטות.



כפי שהוסבר בסעיף הקודם, נוירון E הוא נוירון מסוג Bursting neuron (ירי בצורות). ניתן לראות זאת ב - Raster Plot בכך שהספייקים אינם מפוזרים באופן אחיד לאורך הזמן, אלא מופיעים בקבוצות צפופות של ירי מהיר, שביניהן קיימות תקופות שקטות יחסית. בנוסף, תוצאה זו תואמת גם לערך ה - Fano Factor שקיבלנו עבור נוירון E ($FF \approx 6.65$), המעיד על שונות גבוהה במספר הספייקים בין חלונות זמן שונים.

G.1 היסטוגרמת PSTH

```
# normalization settings
neurons_psth = ['A', 'D']
num_trials = 100
num_bins = 200

# bin width in seconds (5 ms)
bin_width_sec = (1000 / num_bins) / 1000

for neuron in neurons_psth:

    # get current neuron trials
    neuron_trials = spike_data[df3['neuron'] == neuron].values

    # keep only first second
    first_second = neuron_trials[:, :new_HZ]

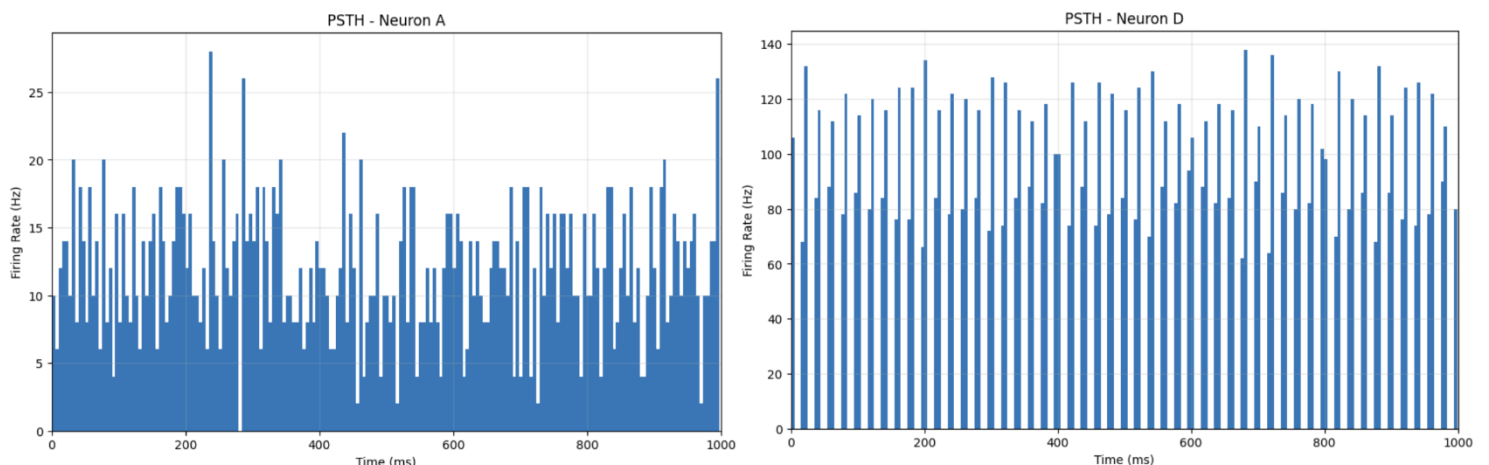
    # collect spike times from all trials
    histogram_data = []

    for trial in first_second:
        spike_positions = np.where(trial == 1)[0]
        histogram_data += list(spike_positions)

    # normalize histogram to firing rate (Hz)
    weights = np.ones_like(histogram_data) / (num_trials * bin_width_sec)
```

בסעיף זה יצרנו גרף PSTH עבור הנוירונים A ו - D במהלך השנייה הראשונה של ההקלטה, כאשר ציר ה - Y מנורמל ל - Firing Rate ביחידות Hz. בקוד נבחרו כל ה - trials של כל נוירון, ומתוך כל trial נאספו זמני הספייקים באמצעות np.where. לאחר מכן אוחדו כל זמני הירי לרשימה אחת ונבנתה היסטוגרמה עם 200 bins, כך שכל bin מייצג חלון זמן של 5 ms. כדי להמיר את מספר הספייקים לקצב ירי ב - Hz, בוצע נרמול באמצעות חלוקת כל ספייק במספר ה - trials וברוחב ה - bin בשניות (weights).

מהתוצאות ניתן לראות כי נוירון A מציג פעילות יחסית אחידה לאורך הזמן, ללא פיקים חדים, דבר המתאים לירי אקראי פואסוני. לעומת זאת, נוירון D מציג פיקים גבוהים וחדים בזמנים מסוימים, כלומר ספייקים רבים מופיעים באותו חלון זמן במספר רב של trials. לכן מתקבלים ערכי firing rate רגועים גבוהים. הסיבה לכך שהפיקים בנוירון D גבוהים יותר מקצב הירי הממוצע היא שה - PSTH מודד קצב ירי רגועי בתוך חלונות זמן קצרים, ולא ממוצע על פני כל ההקלטה. כאשר הרבה trials מסונכרנים לאותו זמן, מתקבל פיק גבוה גם אם קצב הירי הממוצע הכולל של הנוירון נמוך יותר.



Autocorrelation Plot A.2

```
duration_2_min_ms = 2 * 60 * 1000
max_lag_ms = 100

neurons_to_analyze = ['A', 'C', 'D', 'E']

for neuron_name in neurons_to_analyze:

    # get all trials of current neuron
    neuron_trials = spike_data[df3['neuron'] == neuron_name].values

    # concatenate trials into one long signal
    flattened_signal = neuron_trials.flatten()

    # keep first 2 minutes
    signal_segment = flattened_signal[:duration_2_min_ms]

    # compute autocorrelation
    corr = np.correlate(signal_segment, signal_segment, mode='full')

    mid_point = len(corr) // 2

    # create lag axis in ms
    lags = np.arange(-max_lag_ms, max_lag_ms + 1)

    # keep only small lag window around zero
    corr_segment = corr[
        mid_point - max_lag_ms : mid_point + max_lag_ms + 1
    ].astype(float)

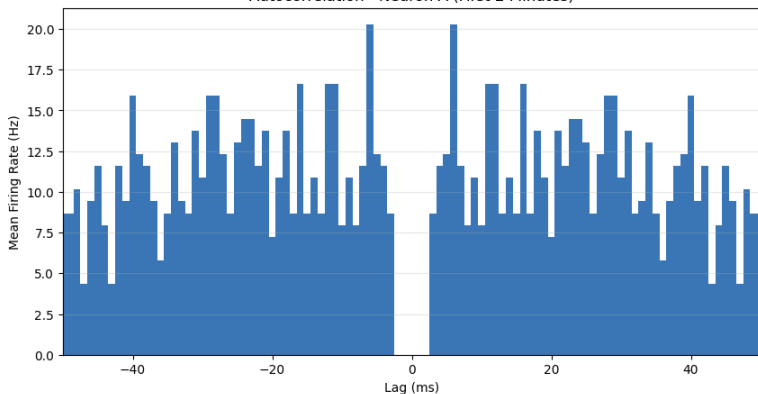
    # normalize to mean firing rate
    num_spikes = np.sum(signal_segment)
    p_firing = corr_segment / num_spikes
    fr_hz = p_firing * new_HZ

    # remove self-correlation peak at lag 0
    fr_hz[max_lag_ms] = 0
```

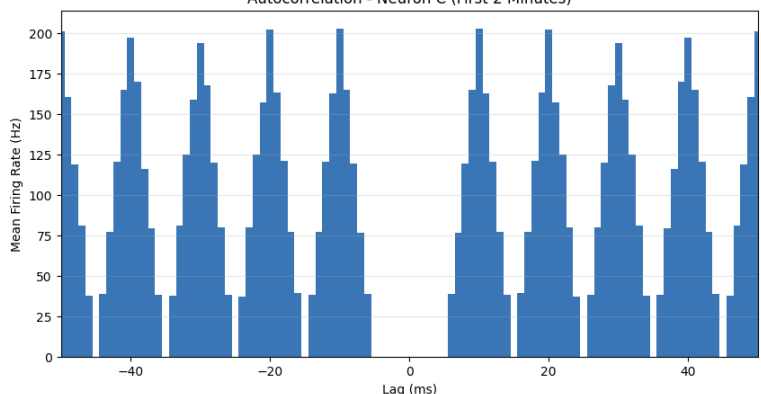
בסעיף זה חישבנו והצגנו את האוטוקורלציה עבור הניורונים A, C, D ו-E על פני שתי הדקות הראשונות של ההקלטה. בקוד אוחדו כל ה-trials של כל ניורון לרצף זמן ארוך אחד, ומתוכו נלקחו 120,000 דגימות המתאימות ל-2 דקות בתדר דגימה של 1000 Hz. לאחר מכן חישבנו את האוטוקורלציה באמצעות np.correlate. כדי לנרמל את התוצאה ל-Mean Firing Rate, חילקנו את מספר ההתאמות במספר הספייקים הכולל ולאחר מכן הכפלנו בתדר הדגימה. בנוסף, הערך ב lag=0 אופס.

מהגרפים ניתן לראות דפוסי ירי שונים בין הניורונים. ניורון A מציג אוטוקורלציה יחסית שטוחה סביב האפס. בגרף מופיע קו שטוח יחסית מיד לאחר התקופה הרפקטורית, דבר האופייני לירי פואסוני ללא מחזוריות. ניורון C מציג פיקים מחזוריים וסדירים במרווחים קבועים. ניורון D מציג גם הוא מבנה מחזורי, אך עם שונות מעט גבוהה יותר ולכן הפיקים מרווחים יותר. לעומת זאת, ניורון E מציג פיק חזק סביב lag=0, המעיד על Burst firing. בנוסף, סמוך מאוד ל-lag=0 ניתן לראות ירידה חדה בפעילות, המייצגת את התקופה הרפקטורית.

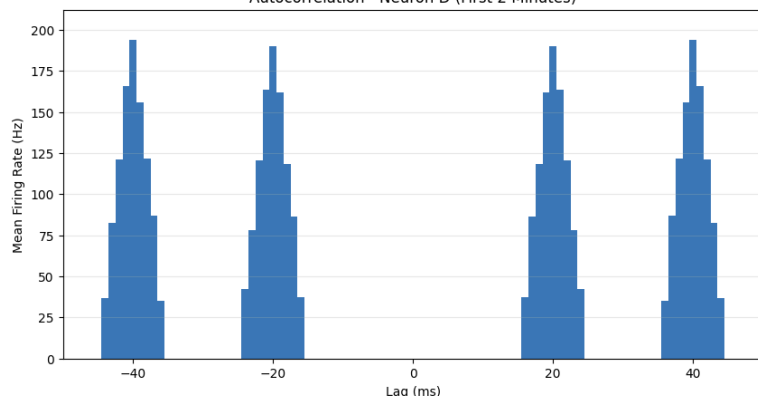
Autocorrelation - Neuron A (First 2 Minutes)



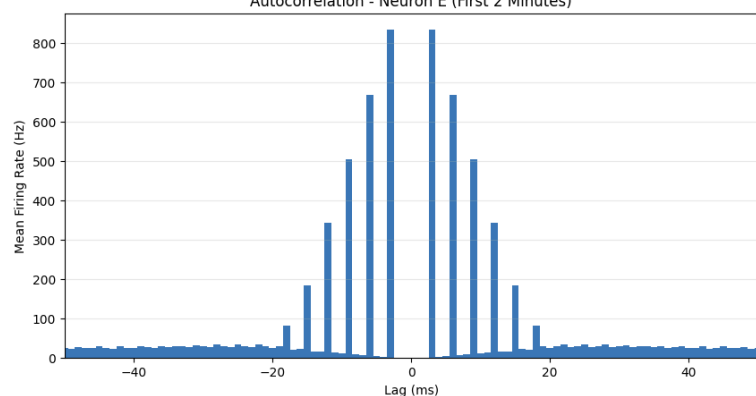
Autocorrelation - Neuron C (First 2 Minutes)



Autocorrelation - Neuron D (First 2 Minutes)



Autocorrelation - Neuron E (First 2 Minutes)



B.2 Autocorrelation – סיווג נירון B

כפי שראינו בסעיף הקודם, נירון C הוא נירון רגולרי. ניתן לראות זאת בגרף האוטוקורלציה, שבו מופיעים פיקים מחזוריים חדים במרווחי זמן קבועים סביב $\text{lag}=0$. פיקים אלו מעידים שהנירון נוטה לירות במרווחי זמן דומים בצורה מחזורית. בנוסף, סביב $\text{lag}=0$ ניתן לראות ירידה חדה המתאימה לתקופת הרפרקטוריות, שבה הנירון אינו יכול לירות מיד לאחר ספייק קודם.

ערך ה-Fano Factor שקיבלנו עבור נירון C היה $\text{FF}=0$, והוא אכן תואם את ההשערה שמדובר בנירון רגולרי. ערך זה מעיד שכמעט ואין שונות במספר הספייקים בין ה-trials השונים, כלומר הנירון יורה בצורה כמעט קבועה לחלוטין. לכן קיימת התאמה מלאה בין דפוס האוטוקורלציה המחזורי לבין ערך ה-FF.

C.2 Autocorrelation – סיווג נירון C, D

נירונים C ו-D הינם נירונים רגולריים. ההבדל המרכזי בין פונקציות האוטוקורלציה של C ו-D הוא רמת הסדירות של הירי, כאשר נירון D פחות סדיר מנירון C. בנירון C מתקבלות פסגות חדות, גבוהות ומרוחקות במרווחים קבועים. לעומת זאת, בנירון D הפסגות פחות חדות, ומרווחות יותר, כלומר הוא יורה בקצב נמוך יותר. הבדל זה תואם גם לערכי ה-Fano Factor, נירון C קיבל $\text{FF}=0$, בעוד נירון D קיבל ערך מעט גבוה יותר $\text{FF}\approx 0.01$, המעיד על שונות קטנה.

D.2 Autocorrelation – נירון E

ניתן לראות כי סביב $\text{lag}=0$ מתקבל מבנה רחב וסימטרי עם פסגות גבוהות הקרובות לאפס. צורה זו מעידה כי נירון E הוא נירון מסוג bursting כלומר נירון שיורה רצף של ספייקים צפופים בזמן קצר. לאחר הופעת ספייק אחד קיימת הסתברות גבוהה מאוד להופעת ספייקים נוספים, ולכן מתקבלים ערכי קורלציה גבוהים במיוחד בערכי lag קטנים סביב האפס.

הצורה המשולשת הרחבה סביב האפס משקפת את משך ה-burst ככל שמתרחקים מ- $\text{lag}=0$ ההסתברות למצוא ספייק נוסף יורדת בהדרגה ולכן גם גובה העמודות קטן. מחוץ לאזור זה הקורלציה חוזרת כמעט לערך הבסיס של קצב הירי הממוצע.

1. מספר הפיקים בגרף האוטוקורלציה יכול להעיד על דפוס הירי והמחזוריות של הנירון. המרחק בין הפיקים מייצג את מרווח הזמן האופייני בין ספייקים (ISI) ולכן גם קשור לתדר הירי. כאשר מופיעים פיקים מחזוריים במרווחים קבועים, הדבר מצביע על ירי סדיר. לעומת זאת, אם מתקבלת בעיקר פסגה מרכזית ללא פיקים מחזוריים נוספים, הדבר מעיד על ירי אקראי, כמו בתהליך פואסוני, שבו אין מחזוריות ברורה. בנירונים מסוג bursting ניתן לראות מספר פיקים קרובים מאוד סביב ($\text{lag}=0$), המעידים על הופעת כמה ספייקים צפופים בזמן קצר בתוך כל צרור.
2. המינימום המקומי שמופיע אחרי כל פיק באוטוקורלציה מעיד על כך שלאחר ירי של ספייק קיימת ירידה זמנית בהסתברות לירי נוסף. תופעה זו קשורה בעיקר לתקופה הרפרקטורית.

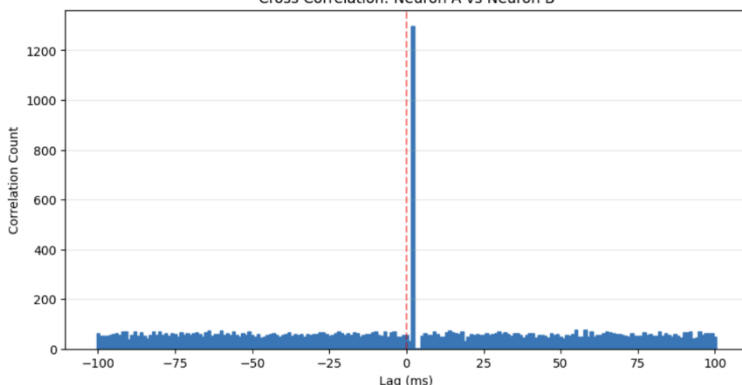
בנוירונים סדירים, המינימום לאחר כל פיק מעיד גם על מחזוריות הירי, לאחר שהנרירון יורה בזמן מסוים, יש "חלון שקט" קצר לפני הספייק הבא במחזור. לכן מתקבל דפוס של פיקים חוזרים. ככל שהמינימום עמוק וברור יותר, כך ניתן להסיק שהירי מדויק וסדיר יותר, עם מרווחי זמן יחסית קבועים בין הספייקים.

Cross – Correlation E.2

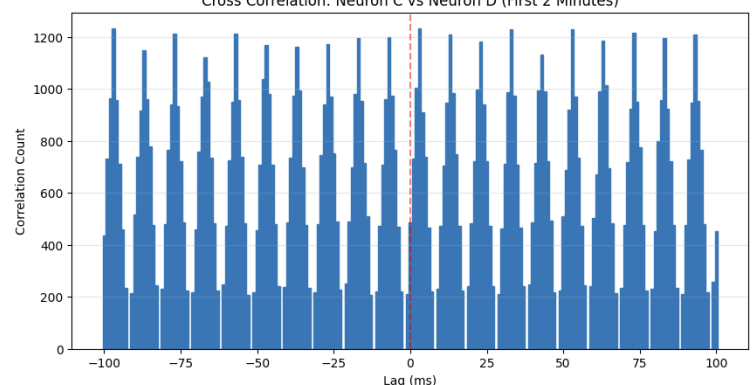
בסעיף זה ביצענו ניתוח קרוס - קורלציה עבור שני זוגות של נוירונים: (A, B) ו-(C, D). לצורך כך, השתמשנו בפונקציה `get_long_recording` המבצעת שיטוח (Flatten) של כל ה- trials לכדי רצף אחד ארוך, ודוגמת את שתי הדקות הראשונות של ההקלטה. חישוב הקורלציה התבצע באמצעות הפונקציה `np.correlate` במצב `'mode=full'`, המחשבת את מספר החפיפות (Spike counts) בין שני הסיגנלים עבור כל הזזה (Lag) אפשרית. בהתאם לדרישות, הצגנו את התוצאות ללא נרמול, תוך ביצוע "זום" לטווח של כ- 100 ms סביב נקודת האפס כדי לזהות אינטראקציות סינפטיות.

```
def get_long_recording(neuron_label, data_df, limit_samples):  
  
    # extract trials of selected neuron  
    trials = data_df[data_df['neuron'] == neuron_label].drop(  
        columns=['trial_neuron', 'neuron', 'trial_fr', 'spike_count'],  
        errors='ignore').values  
  
    # create one long spike train  
    flattened = trials.flatten()  
  
    return flattened[:limit_samples]  
  
pairs = [('A', 'B'), ('C', 'D')]  
  
for n1, n2 in pairs:  
  
    # get 2-minute recordings  
    sig1 = get_long_recording(n1, df3, duration_2_min_ms)  
    sig2 = get_long_recording(n2, df3, duration_2_min_ms)  
  
    # compute cross-correlation  
    cc_full = np.correlate(sig2, sig1, mode='full')  
  
    lags = np.arange(-(len(sig1) - 1), len(sig1))  
    center_idx = len(cc_full) // 2  
  
    # zoom around lag=0  
    cc_zoom = cc_full[center_idx - max_lag_ms : center_idx + max_lag_ms + 1]  
  
    lags_zoom = lags[center_idx - max_lag_ms : center_idx + max_lag_ms + 1]
```

Cross Correlation: Neuron A vs Neuron B



Cross Correlation: Neuron C vs Neuron D (First 2 Minutes)



1. בזוג הניורונים A ו-B ניתן לראות בגרף הקרוסקורלציה שיא מעל רמת הרקע, דבר המעיד על קשר זמני בין הניורונים ולא על ירי אקראי בלתי תלוי. מאחר שמדובר בשיא חיובי ולא בירידה, הקשר הוא קשר אקסיטטורי, כלומר ירי של ניורון A מגדיל את ההסתברות שניורון B יירה מיד אחריו. מיקום הפיק ב Lag חיובי של כ- 3 ms מצביע על כך שניורון A יורה ראשון ולכן הוא הניורון המפעיל, בעוד ניורון B מגיב לאחר השהייה סינפטית קצרה. בנוסף, רמת הרקע בגרף יחסית שטוחה משום שניורון A מציג ירי פואסוני אקראי ללא מחזוריות קבועה.
- בין הניורונים C ו-D מתקבלת סדרה של פיקים מחזוריים במרווחים קבועים של כ- 3 ms , ללא הופעה של פיק דומיננטי יחיד סביב lag מסוים. תבנית זו אינה אופיינית לקשר ניורוני ישיר, שבו מצופה לראות שיא ברור המעיד על תלות זמנית עקבית בין ירי הניורונים. במקום זאת, המחזוריות הקבועה מצביעה ככל הנראה על קצב ירי דומה של שני הניורונים, או השפעה של קלט חיצוני משותף. לכן, לא נמצאה עדות מובהקת לקשר סינפטי או פונקציונלי ישיר בין הניורונים על סמך התוצאות.
2. אמפליטודת הפיק בגרף הקרוסקורלציה (שאינו מנורמל) מייצגת את מספר המופעים המשותפים שבהם ניורון B ירה בטווח זמן מוגדר לאחר הירי של ניורון A. גובה השיא מעיד על חוזק הקשר התפקודי בין התאים ועל מידת המהימנות שבה ניורון הפרה - סינפטי מצליח להפעיל את הניורון הפוסט - סינפטי. תחת ההנחה שכל ספייק של ניורון A גורם לירי של ניורון B, אמפליטודת השיא משקפת למעשה את סך כל הספייקים שירה ניורון A במהלך כל תקופת ההקלטה. כדי להעריך את קצב הירי של ניורון A ביחידות של Hz , נשתמש בערך המקסימלי של הפיק ונחלק אותו במשך זמן ההקלטה שנבדק (במקרה זה 120 שניות). שיטה זו מאפשרת לנו להסיק מידע על קצב הפעילות של ניורון אחד מתוך התבוננות בקורלציה שלו עם ניורון אחר, ובלבד שהקשר הסינפטי ביניהם חזק וידוע.
3. אם ניורון A היה מפעיל את ניורון B בהסתברות של 50% בלבד, גובה הפיק בקרוסקורלציה היה קטן פי 2. הסיבה היא שלא כל ספייק של ניורון A היה מוביל לספייק תואם בניורון B, אלא רק חצי מהם בממוצע. לכן מספר ההתאמות סביב זמן ההשהיה היה קטן, והפיק היה נמוך יותר אך עדיין מופיע באותו lag . כלומר, מיקום הפיק לא משתנה, אלא רק האמפליטודה שלו קטנה, משום שהקשר בין הניורונים נעשה פחות אמין.

Cross – Correlation G.2 - עם נרמול

בתרגיל זה חישבנו והצגנו Cross-Correlation בין הניורונים C ו-D עבור שתי הדקות הראשונות של ההקלטה, במטרה לבדוק תלות זמנית ביניהם. הקרוסקורלציה חושבה באמצעות `np.correlate`, ולאחר מכן בוצע נרמול כך שהתוצאה מייצגת קצב ירי מותנה ביחידות Hz , כלומר הסתברות יחסית לירי של ניורון D בהינתן ירי של ניורון C. בגרף, ציר ה- x מייצג lag וציר ה- y מייצג את עוצמת הקשר לאחר נרמול. מהגרף ניתן לראות כי אין שיא חד וברור סביב $\text{lag} = 0$, ולכן לא מתקבל סנכרון בזמן אמת בין C ל-D. בנוסף, הפונקציה יחסית סימטרית סביב האפס, מה שמרמז שאין ניורון שמקדים באופן עקבי את השני. עם זאת, קיימת פעילות גבוהה יחסית לאורך כל תחום lags עם דפוס מחזורי, מה שיכול להעיד על פעילות משותפת כללית או השפעה של גירוי חיצוני, אך לא על קשר זמני ישיר בין שני הניורונים.

```

n1, n2 = 'C', 'D'

# extract first 2 minutes of spike trains
sig1 = get_long_recording(n1, df3, duration_2_min_ms)
sig2 = get_long_recording(n2, df3, duration_2_min_ms)

# raw cross-correlation (unnormalized)
cc_raw = np.correlate(sig1, sig2, mode='full')

# number of spikes in reference neuron (normalization factor)
n_spikes_source = np.sum(sig1)

# convert correlation to conditional firing rate (Hz)
cc_normalized = (cc_raw / n_spikes_source) / bin_width_sec

# create lag axis
lags = np.arange(-(len(sig1) - 1), len(sig1))
center_idx = len(cc_raw) // 2

# zoom around zero lag
cc_zoom = cc_normalized[center_idx - max_lag_ms : center_idx + max_lag_ms + 1]
lags_zoom = lags[center_idx - max_lag_ms : center_idx + max_lag_ms + 1]

```

